

DemoSpacyPropp

January 25, 2026

1 SPACY et Universal Dependencies

Illustration de Spacy, exemples tiré de Thérèse Desqueyroux (Mauriac)

https://ia601700.us.archive.org/10/items/isbn_9789200325229/isbn_9789200325229.pdf

```
[1]: # doc spacy
# https://spacy.io/usage/linguistic-features

import spacy
from spacy import displacy
nlp = spacy.load('fr_core_news_md')
import pandas as pd
```

```
[2]: myTxt = "L'avocat ouvrit une porte."
```

```
[3]: doc = nlp(myTxt)
type(doc)
```

```
[3]: spacy.tokens.doc.Doc
```

```
[4]: displacy.render(doc, style='dep', jupyter=True, options={'distance': 100})
```

<IPython.core.display.HTML object>

```
[5]: for token in doc:
    print("{0}\t{1}\t{2}\t{3}\t{4}\t{5}\t{6}\t{7}\t{8}".format(
        token.text,
        token.i,
        token.lemma_,
        token.is_punct,
        token.is_space,
        token.shape_,
        token.pos_,
        token.dep_,
        token.head.i
    ))
```

L'	0	l'	False	False	X'	DET	det	1
avocat	1	avocat	False	False	xxxx	NOUN	nsubj	2

ouvrit	2	ouvrir	False	False	xxxx	VERB	ROOT	2
une	3	un	False	False	xxx	DET	det	4
porte	4	porte	False	False	xxxx	NOUN	obj	2
.	5	.	True	False	.	PUNCT	punct	2

```
[6]: # donne tous les patrons nominaux
doc.noun_chunks
```

```
[6]: <_cython_3_2_1.generator at 0x74598b668190>
```

```
[7]: for chunk in doc.noun_chunks:
      print(chunk.text, chunk.root.text, chunk.root.dep_,
            chunk.root.head.text)
```

```
L'avocat avocat nsubj ouvrit
une porte porte obj ouvrit
```

```
[8]: for token in doc:
      print(token.text, token.dep_, token.head.text, token.head.pos_,
            [child for child in token.children])
```

```
L' det avocat NOUN []
avocat nsubj ouvrit VERB [L']
ouvrit ROOT ouvrit VERB [avocat, porte, .]
une det porte NOUN []
porte obj ouvrit VERB [une]
. punct ouvrit VERB []
```

```
[9]: root = [token for token in doc if token.head == token][0]
      root.lefts

      subject = list(root.lefts)[0]

      for descendant in subject.subtree:
          assert subject is descendant or subject.is_ancestor(descendant)
          print(descendant.text, descendant.dep_, descendant.n_lefts,
                descendant.n_rights,
                [ancestor.text for ancestor in descendant.ancestors])
```

```
L' det 0 0 ['avocat', 'ouvrit']
avocat nsubj 1 0 ['ouvrit']
```

```
[10]: # Ajout d'un modifieur (adj) à droite et à gauche du nom 'avocat'
myTxt = "Le grand avocat français ouvrit une porte."

doc = nlp(myTxt)

root = [token for token in doc if token.head == token][0]
root.lefts
```

```

# on prend l'elem à gauche du root
subject = list(root.lefts)[0]

# pour cet élément (donc ici le sujet)
# ...on regarde si il a des descendant à droite et à gauche
for descendant in subject.subtree:
    print(descendant.text, descendant.dep_, descendant.n_lefts,
          descendant.n_rights,
          [ancestor.text for ancestor in descendant.ancestors])

```

```

Le det 0 0 ['avocat', 'ouvert']
grand amod 0 0 ['avocat', 'ouvert']
avocat nsubj 2 1 ['ouvert']
français amod 0 0 ['avocat', 'ouvert']

```

```
[11]: displacy.render(doc, style='dep', jupyter=True, options={'distance': 100})
```

```
<IPython.core.display.HTML object>
```

```
[12]: # on prend l'elem à gauche du root
myObject = list(root.rights)[0]

# pour cet élément (donc ici le sujet)
# ...on regarde si il a des descendant à droite et à gauche
for descendant in myObject.subtree:
    print(descendant.text, descendant.dep_, descendant.n_lefts,
          descendant.n_rights,
          [ancestor.text for ancestor in descendant.ancestors])

```

```

une det 0 0 ['porte', 'ouvert']
porte obj 1 0 ['ouvert']

```

```
[13]: # parcourir l'arbre
for token in doc :
    print("-----")
    print("TOKEN :: "+str(token.text))
    print("TETE (token) :: "+str(token.head.text))
    print(token.text, token.dep_, token.head.text, token.head.pos_,
          [child for child in token.children])

```

```

-----
TOKEN :: Le
TETE (token) :: avocat
Le det avocat NOUN []
-----
TOKEN :: grand
TETE (token) :: avocat
grand amod avocat NOUN []

```

```

-----
TOKEN :: avocat
TETE (token) :: ouvrit
avocat nsubj ouvrit VERB [Le, grand, français]
-----
TOKEN :: français
TETE (token) :: avocat
français amod avocat NOUN []
-----
TOKEN :: ouvrit
TETE (token) :: ouvrit
ouvrit ROOT ouvrit VERB [avocat, porte, .]
-----
TOKEN :: une
TETE (token) :: porte
une det porte NOUN []
-----
TOKEN :: porte
TETE (token) :: ouvrit
porte obj ouvrit VERB [une]
-----
TOKEN :: .
TETE (token) :: ouvrit
. punct ouvrit VERB []

```

```

[14]: for token in doc:
    print("{0}\t{1}\t{2}\t{3}\t{4}\t{5}\t{6}\t{7}\t{8}".format(
        token.text,
        token.idx,
        token.lemma_,
        token.is_punct,
        token.is_space,
        token.shape_,
        token.pos_,
        token.tag_,
        token.ent_type_
    ))

```

Le	0	le	False	False	Xx	DET	DET		
grand	3	grand	False	False	xxxx	ADJ	ADJ		
avocat	9	avocat	False	False	xxxx	NOUN	NOUN		
français	16	français	False	False	xxxx	ADJ	ADJ		
ouvrit	25	ouvrir	False	False	xxxx	VERB	VERB		
une	32	un	False	False	xxx	DET	DET		
porte	36	porte	False	False	xxxx	NOUN	NOUN		
.	41	.	True	False	.	PUNCT	PUNCT		

```
[15]: myDebRoman = "L'avocat ouvrit une porte. Thérèse Desqueyroux, dans ce couloir_
↳dérobé du Palais de justice, sentit sur sa face la brume et profondément,
↳l'aspira. Elle avait peur d'être attendue, hésitait a sortir. Un homme, dont_
↳le col était relevé, se détacha d'un platane ; elle reconnut son père. "
docDebRoman = nlp(myDebRoman)
```

```
[16]: for sent in docDebRoman.sents:
      print(sent)
```

L'avocat ouvrit une porte.
 Thérèse Desqueyroux, dans ce couloir dérobé du Palais de justice, sentit sur sa
 face la brume et profondément, l'aspira.
 Elle avait peur d'être attendue, hésitait a sortir.
 Un homme, dont le col était relevé, se détacha d'un platane ;
 elle reconnut son père.

```
[17]: for chunk in docDebRoman.noun_chunks:
      print(chunk.text, " --> ", chunk.label_)
```

L'avocat --> NP
 une porte --> NP
 Thérèse Desqueyroux, dans ce couloir dérobé du Palais de justice, sentit sur sa
 face la brume et profondément, l'aspira. --> NP
 Elle --> NP
 peur --> NP
 Un homme --> NP
 dont --> NP
 un platane --> NP
 elle --> NP
 son père --> NP

```
[18]: for ent in docDebRoman.ents:
      print(ent.text, ent.label_)
```

Thérèse Desqueyroux PER
 Palais de justice LOC

```
[ ]:
```